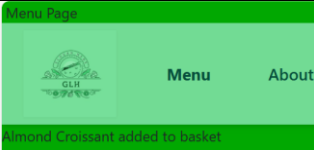
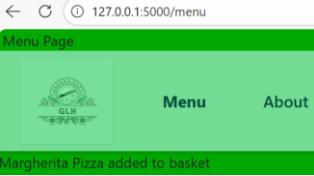
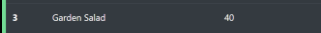
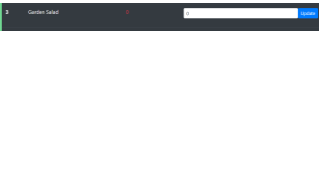
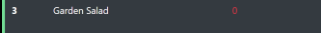
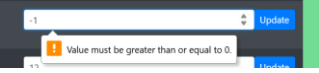
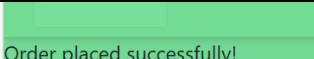
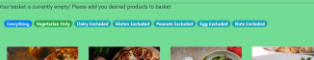
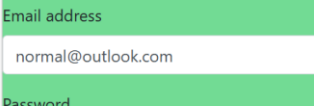


Task 2: Test log

Normal/Boundary/Erroenas testing

Description of test	Test data to be used (if required)	Expected outcome	Actual outcome	Comments and intended actions
Test1- Test if ADD TO BASKET route is accepts normal data	Normal - 9	Almond Croissant added to basket		When using the add_to_basket route and have a stock_id (in my case: 9) a stock item should be added to the basket which way displayed as intended.
Test1- Test if ADD TO BASKET route is accepts boundary data	Boundary - 1	Margherita Pizza added to basket		When using the add_to_basket route and have a stock_id (in my case: 9) a stock item should be added to the basket which way displayed as intended.
Test2- Test if a admin can enter a normal data input in stock_management	Normal - 5 	Spicy Pepperoni Pizza Stock Level should change to 5		System successfully altered stock_level to the desired amount with no issues

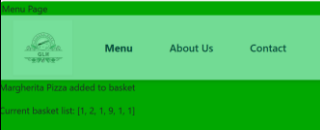
Test2- Test if a admin can enter a boundary data input in stock_management	Boundary-0 	Garden Salad Stock Level should change to 0		System successfully altered stock_level to the desired amount with no issues
Test2- Test if a admin can enter a erroneous data input in stock management	Erroneous- -1 	Garden Salad Stock Level should remain at 0		System successfully doesn't allow stock level to be less than 0
Test2- Test if a admin can enter a erroneous data input in stock management	Erroneous- 	Garden Salad Stock Level should change to the desired amount		System allows a large amount of stock to be added to the stock level. The producers may not have that amount of stock available. This is why its paramount administrator access must remain secure so inflated stock levels with no occur
Test3- Test if checkout accepts a normal subtotal	Normal- £7.50	Placing a Order with a acceptable amount of subtotal should result in a flash message signally the user order has gone through	KeyError KeyError: 'user_id'	Unexpected Error, current system demands for user to be logged in when placing a order. FIX: Change nullable in order class from False to True

Test3- Test if checkout accepts a normal subtotal	Normal- £7.50	Placing a Order with a acceptable amount of subtotal should result in a flash message signally the user order has gone through		Test success- System now allows non logged in users place orders. First most test also accepted the normal data no problem
Test3- Test if checkout accepts erroneous subtotal	Erroneous - £0.00 	Placing a order with no subtotal should simply return user back to menu page for them to add their desired items to basket		Test Successful- System successfully redirected the user and flashed the firm message with no issues
Test4- Test if login/register route accept normal logins	Normal- normal@outlook.com	No in-built message should pop up asking the email for required char		Test successful, no warning messages
Test4- Test if login/register route accept Erroneous logins	Erroneous- normaloutlook.com	No in-built message should pop up asking the email for required char		Test successful, warning message successfully popped up

Black/White Box Testing

ADD TO BASKET Testing-

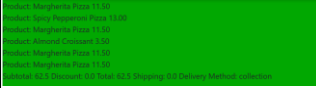
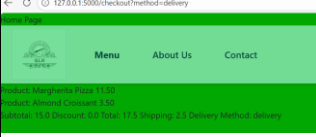
Description of test	Testing Method	Test data to be used (if required)	Expected outcome	Actual outcome	Comments and intended actions
---------------------	----------------	------------------------------------	------------------	----------------	-------------------------------

Test- Test if /add_to_basket/ route is adding to basket	White Box	<pre> app.route('/add_to_basket/stock_id') def add_to_basket(): # Initialize basket as a list if it doesn't exist if "basket" not in session: session["basket"] = [] # Querying for the SPECIFIC item using the ID item = Stock.query.get(stock_id) if item and item.stock_level > 0: # Get the list, updates it, then re-saves it basket = session["basket"] basket.append(item) # Storing the stock ID only session["basket"] = basket # Turning basket back withing a session # Flashing the message that a stock item has been added to basket flash(f"{item.stock_name} added to basket") else: flash(f"Item out of stock or not found") return redirect(url_for("menu")) </pre>	Item should be added to empty basket	No actual Outcome- White Box	Algorithm is logical and operates correctly but there is no indication if the item is actual adding to the basket, just a mere flashed message! We are going to alter the code to test if the basket is really being altered
Test- if /add_to_basket/ route is adding to basket	Black Box	<pre> # Flashing the message that a stock item has flash(f"{item.stock_name} added to basket") flash(f"Current basket list: {basket}") else: flash(f"Current basket list: {basket}") flash(f"Item out of stock or not found") </pre>	Added items should be listed in a flashed message		The actual stock id of the items was listed in the list as I intended signifying the basket is actual adding the product.

CHECKOUT Testing-

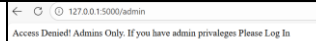
Description of test	Testing Method	Test data to be used (if required)	Expected outcome	Actual outcome	Comments and intended actions
Test1- Testing if basket exists withing checkout route	White Box	<pre> app.route('/checkout') def checkout(): # 1. Get the basket or an empty list if it's missing basket_ids = session.get("basket", []) subtotal = 0 display_items = [] </pre>	Basket should be listed withing checkout URL	No actual Outcome- White Box	Basket should be displayed properly without much trouble

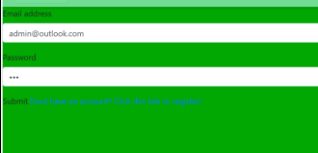
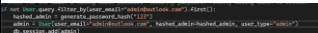
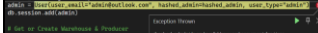
		<pre> <div class="checkout_items"> {% for item in items %} <p> {{item.stock_name}} </p> {% endfor %} <p> Basket: {{basket}} Subtotal: {{subtotal}} Discount: {{discount}} Total: {{total}} Shipping: {{shipping}} Delivery Method: {{method}} </p> </div> </pre>			
Test1- Testing if basket exists withing checkout route	Black Box	/checkout route	Basket should be listed withing checkout url	<pre> Basket: None Subtotal: None Discount: None Total: None Shipping: None Delivery Method: None </pre>	Basket was listed properly but they are key data aspects missing I'm sure the user will appreciate. CODE CHANGE NEEDED
Test2 - Make item price visible to the users also so they don't have to calculate for themselves	Black Box	<pre> <p> Product: {{item.stock_name}} {{item.stock_price}} </p> {% endfor %} </pre>	Both stock_name and stock_price should be outputted	<pre> Basket: None Subtotal: None Discount: None Total: None Shipping: None Delivery Method: None </pre>	Stock price successfully included
Test3- Test if delivery method is fully functional	White Box	<pre> # Providing the acquiring of order solution GLH asked method = request.args.get("method", "collection") shipping_fee = 2.50 if method == "delivery" else 0.00 grand_total = discounted_subtotal + shipping_fee # 3. Pass the correct variables to the template return render_template("checkout.html", items=display_items, subtotal=subtotal, discount=loyalty.discount, total=grand_total, shipping=shipping_fee, method=method) </pre>	Shipping should be 0.0 & delivery method should be collection	No actual Outcome- White Box	As set in both methods & shipping fee they should be 0 in shipping fee and delivery_method should remain collection

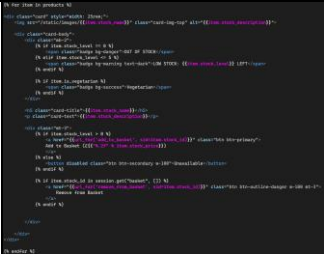
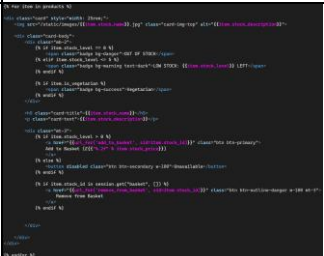
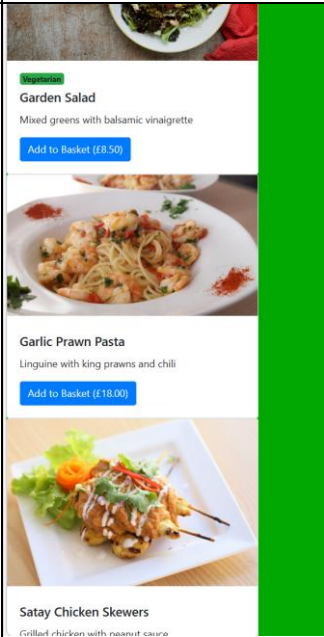
Test3- Test if delivery method is fully functional	Black Box	/checkout route	Shipping should be 0.0 & delivery method should be collection		Both elements remain as default; we will now see if the algorithm works when the method is delivery.
Test3- Test if delivery method is fully functional	Black Box	127.0.0.1:5000/checkout/method=delivery 127.0.0.1:5000/checkout/method=delivery	Shipping should be 2.5 & delivery method should be delivery		Test successful but there is a hidden error, even though the shipping_fee is visible it didn't change the subtotal amount. CODE CHANGE NEEDED
Test4- Test if Loyalty Discount operates	White Box	<pre>loyalty_discount = 0.00 discounted_subtotal = 12.50 loyalty_discount = 0.00 discounted_subtotal = 12.50</pre>	Discount should apply if a user is logged in and arithmetically outputted when done	No actual Outcome- White Box	Logic of my algorithm is simple but effective, no problem directly found from first look
Test4- Test if Loyalty Discount operates	Black Box	Logging In , Adding items, to basket, seeing checkout page	Discount should apply if a user is logged in and arithmetically outputted when done		Discount was successfully applied correctly. Arithmetically accurate also.
Test5- Test if total is accurate	White Box	<pre>#tallling up every key value at once grand_total = discounted_subtotal + shipping_fee</pre>	Total should be accurate	No actual Outcome- White Box	Adds both discounted_subtotal (loyalty or not) and the shipping fee (depending on

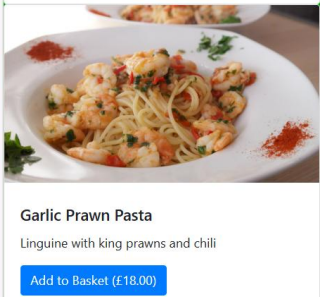
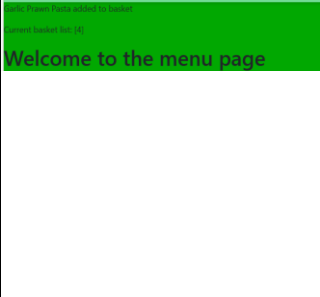
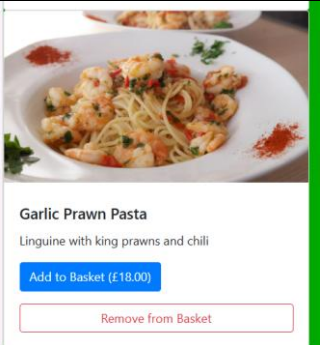
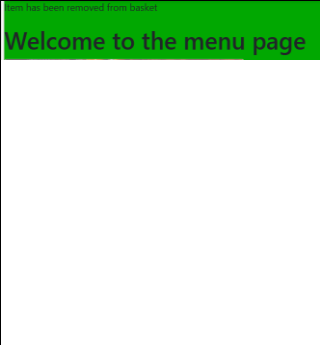
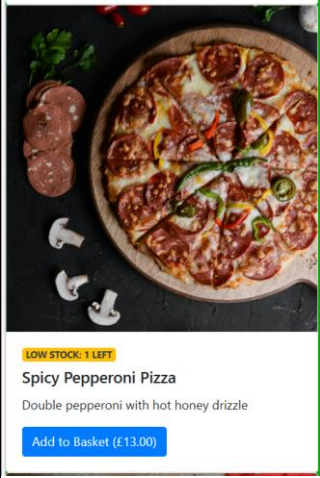
					method). Simple arithmetic should be 100% accurate
Test5- Test if total is accurate	Black Box	/checkout route	Total should be accurate	Product: Merchandise Total: 11.00 Product: Removal Cost: 5.00 Subtotal: 16.00 Discount: 8.00 Total: 16.00 Shipping: 2.0 Delivery Method: Delivery	Fully accurate

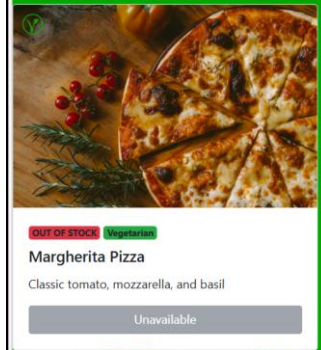
Admin Testing-

Description of test	Testing Method	Test data to be used (if required)	Expected outcome	Actual outcome	Comments and intended actions
Test1- Testing if admin page is accessible the users	White Box	<pre>def admin(): # Returns a simple message and the URL (localhost) return "Access Denied! Admin Only. If you have admin privileges Please Log In", 403</pre>	User should be sent to an error page warning them	No actual Outcome- White Box	Code is concise only using data its needs for now, admins will later along the data solution development be able to add stocks of their own
Test1- Testing if admin page is accessible the users	Black Box	<pre>127.0.0.1:5000/admin 127.0.0.1:5000/admin - 127.0.0.1:5000/admin</pre>	User should be sent to an error page warning them		Test successful, user was sent to the error page and warned as intended
Test2- Test if admin has entered credentials they are sent to the admin	White Box	<pre>def admin(): # Returns a simple message and the URL (localhost) return "Access Denied! Admin Only. If you have admin privileges Please Log In", 403</pre>	Admins with the correct credentials should be sent to the admin page	No actual Outcome- White Box	Code checks the user_type and renders template if so. Should be fully

page					operational
Test2- Test if admin has entered credentials they are sent to the admin page	Black Box		Admins with the correct credentials should be sent to the admin page		Test successful-admin enter admin page with no problem, security test is now needed
Test3- admins password has been hashed	White Box		Password should be hashed in data base to prevent/defend against potential data breaches	No actual Outcome- White Box	Code blocks look secure and functional. Hashing should happen without fault
Test3- admins password has been hashed	Black Box	Running web app	Database should be seeded and password should be hashed		Error Found, system tells me that hashed admin is an invalid keyword for the User table. I have now realised that I didn't include my password_hash attribute and used generate hashing instead. LOGIC ERROR & CODE

Test1- Menu Page Testing if menu is fully operational with item cards newly added	White		All products should be listed		All products were listed but their images didn't render alongside them. Both static & images files exists with my product images inside image. FIX: Their format wasn't taken into consideration, simple syntax error
Test1- Menu Page Testing if menu is fully operational with item cards newly added	White		All products should be listed		Test successful- Images all rendered without fault

Test2- Menu Page Test if adding to basket via card input is functional	Black Box		Product 4 should be added to basket without fault		Test successful- item added to basket without fault
Test3- Menu Page Test if removing from basket via card input is functional	Black Box		Product 4 should be removed from basket without fault		Test successfull
Test4- Menu Page Test if the product card signals the user if a product is low on stock	Black Box	Spicy Pepperoni Pizza	A badge should appear on the card alerting the user		Test successful- A accurate badge appeared on the product Card

Test4- Menu Page Test if the product card signals the user if a product is out of stock	Black Box	Margherita Pizza	A badge should appear on the card alerting the user	 The screenshot shows a product card for 'Margherita Pizza'. At the top left of the card, there is a red badge that says 'OUT OF STOCK' and a green badge that says 'Vegetarian'. Below the image of the pizza, the text reads 'Margherita Pizza' and 'Classic tomato, mozzarella, and basil'. At the bottom of the card, there is a grey button that says 'Unavailable'.	Test successful- A accurate badge appeared on the product Card
---	------------------	-------------------------	--	---	---

Add more rows and tables as required.